

The 5x Rule: Choosing the Third Stage of a Confidence-Gated Model Cascade

A simulation study and a protocol for real models

Giovanni Simonetti

Technical report, draft 0.1, October 2026

Abstract

In a model cascade a small model answers first, and a query moves up to a larger model only when a calibrated confidence stays below a threshold. Recent work shows that two tiers of locally served models save a third or more of the compute at equal accuracy. This report asks the next practical question: when should a cascade have a third stage, and what should that stage be? We compare a middle model and a deterministic rule stage on a synthetic workload, across cost ladders, confidence-signal qualities and shares of easy queries. The result is a rule of thumb, the 5x rule. When neighbouring models are less than 5x apart in cost, a middle model loses compute in 36 of 45 simulated workloads, by up to 24 points, while a rule stage that answers 30% of queries gains up to 18. From 5x upward a middle model pays only when the confidence signal is weak. Every figure is simulated; no real model was run. We publish the code that reproduces each number, a planner and a browser calculator, and a protocol for testing the rule on real models, including what would count against it.

1. Introduction

Running a large language model on every query is wasteful when most queries are easy. A cascade addresses this: a small model answers first, reports how confident it is, and passes the query to a larger model only when that confidence is too low. On local hardware the saving is concrete. It is compute, energy and latency on machines that are already paid for.

The idea is well established. FrugalGPT [1] introduced cascades of language-model APIs. More recent work calibrates the confidence before using it and runs on locally served open models: UCCI [2] reports a 31% cost reduction with two models and isotonic calibration, and Conformal Cascade [3] reports 43% with formal accuracy guarantees. Both are evaluated with two tiers.

A practitioner who has two tiers working faces an obvious next step: add a third. There are two candidates. One is a middle model, between the small and the large. The other is not a model at all: a deterministic stage of exact lookups and fixed business rules that answers what it can before any model runs. The published work gives little guidance on which to choose, or on when to add nothing.

This report offers a first answer, from simulation:

- **A decision rule for the third stage**, the 5x rule, based on the cost ratio between neighbouring models and on the quality of the confidence signal.
- **A deterministic rule stage treated as a stage in its own right**, compared with a middle model on the same queries.
- **A planner** that turns four numbers a team can measure into a shortlist of designs worth testing, available from the command line and in the browser.
- **A real-model protocol**, with the outcomes that would count against the rule stated in advance.

The limits are stated plainly. Cascades, calibrated confidence and routing on local models are prior work, and this report does not claim them. The method was reached independently and then checked against the

literature. All results are simulated on synthetic data, and the rule is a hypothesis to test, not a finding about real systems.

2. Related work

Cascades and routing. FrugalGPT [1] chains model APIs and stops at the first acceptable answer. AutoMix [7] uses self-verification to decide when to escalate. UCCI [2] and Conformal Cascade [3] calibrate the confidence signal, with isotonic regression and conformal prediction respectively, and evaluate on open-weight models served locally. CARGO [4] and confidence tokens [5] study confidence-aware routing more broadly, and [6] examines when models know what they do not know. Conformal Cascade defines a multi-tier framework but evaluates it with two tiers.

Staged elimination. Choosing among many options with a fixed budget by eliminating in stages is the subject of Sequential Halving [8], its generalisations [9, 10] and Hyperband [11], and goes back to multi-stage selection in statistics [12]. The repository applies the same logic to choosing among candidate prompts or configurations; those experiments are summarised in section 7.

Statistics. Calibration by binning and its measurement follow standard practice [13]. The calibration test used here is the chi-square goodness-of-fit test in the form of Hosmer and Lemeshow [14].

3. Setup

3.1 Workload and models

Each query has a latent difficulty d . A share π of queries is easy, with $d \sim N(-2.5, 0.5^2)$; the rest is hard, with $d \sim N(1, 1)$.

Three models have abilities $a = (1.0, 1.8, 2.4)$ for small, medium and large. Model f answers correctly when $a_f - d + \varepsilon > 0$, with $\varepsilon \sim N(0, 1)$. With 70% easy queries the three models alone reach 85%, 91% and 95% accuracy.

Each model also emits a raw confidence signal

$$q_f = (a_f - d) + sz, \quad z \sim N(0, 1).$$

The noise s sets the quality of the signal: 0.5 is called good, 1.0 medium, 1.5 weak. In terms a team can measure, with 70% easy queries the small model’s signal separates its right answers from its wrong ones with an area under the ROC curve of about 0.96, 0.94 and 0.91.

Costs are relative compute per query. To study the distance between models with one parameter, most experiments use ladders of the form $1 : r : r^2$, so that r is the cost ratio between neighbouring models.

3.2 Calibration and routing

The raw signal is never used directly. On a calibration sample it is split into 200 quantile bins, and the calibrated confidence of a bin is the observed frequency of correct answers in it.

Stages run in order. A stage’s answer is accepted when its calibrated confidence reaches a threshold θ ; otherwise the query moves on. The last stage always answers. The threshold is chosen from a fixed grid as the value with the lowest mean cost on the calibration sample, among those that keep accuracy within half a point of the large model alone.

All reported figures come from an independent test sample of 300,000 queries. Saving is $1 - \text{cost}/c_{\text{large}}$: the share of compute saved against running the large model on everything.

3.3 The rule stage

A rule stage runs before the models. It answers a share of all queries, the coverage, and passes on the rest. In the simulation it is idealised: it fires only on easy queries, it is 99.5% accurate on what it answers, and it costs 1% of the small model’s compute, paid by every query.

3.4 Designs compared

The reference is the large model alone. The two-stage design is small then large. The three-stage designs are small, medium, large, and rules, small, large. Every comparison uses the same simulated queries for all designs, and gains are reported in points of saving against the two-stage design.

4. Results

4.1 The workload sets the saving

Table 1 shows the saving with a wide cost ladder. The share of easy queries decides most of it: with no easy queries a cascade saves almost nothing or costs more, with 90% it saves 77% to 92%. The method cannot create a saving the workload does not contain.

Table 1: Compute saved against the large model alone, costs
1 : 8 : 70, accuracy within half a point.

Easy queries	Good signal, 3 stages	Good signal, 2 stages	Weak signal, 3 stages	Weak signal, 2 stages
0%	5%	4%	-6%	1%
50%	52%	54%	38%	34%
70%	73%	72%	60%	48%
90%	92%	90%	85%	77%

As a sanity check, at the cost ratio of published two-tier studies (1 : 2.7) the simulation gives 12% to 55% with 70% to 90% easy queries. The 31% and 43% reported on real models [2, 3] fall inside that range. The simulation does not claim better results; it only shows that its numbers are of a plausible size.

4.2 When a middle model pays

Figure 1 shows what adding a middle model does, as the cost ratio between neighbouring models goes from 2x to 10x.

With a good or medium signal the middle model is close to useless: across all ratios it moves the saving by -24 to +4 points, and below 5x it never gains more than about a point. With a good signal the small model already knows which queries to pass on, so an intermediate stop adds cost and little else.

With a weak signal the picture changes. The small model escalates many queries it could not judge, and a middle model catches part of them at a fraction of the large model’s cost. The mean gain turns positive between 3x and 4x and reaches about 10 points at 10x. From 5x upward the middle model never loses more than a fraction of a point in these simulations, and gains up to 15.

Below 5x, across the three signal qualities, the middle model loses half a point or more in 36 of 45 cells and gains in 6. All six gains are at 3x or 4x: five with a weak signal, one with a good signal and 90% easy queries.

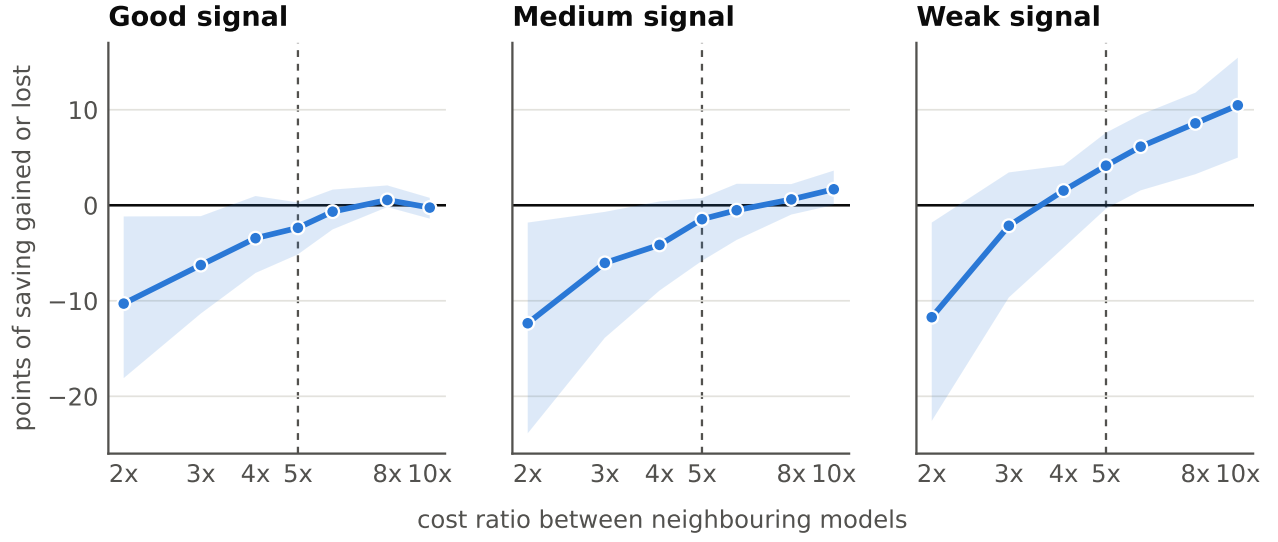


Figure 1: Points of saving gained or lost by adding a middle model to a two-stage cascade. Line: mean over five shares of easy queries (50% to 90%). Band: minimum to maximum. Dashed line: 5x.

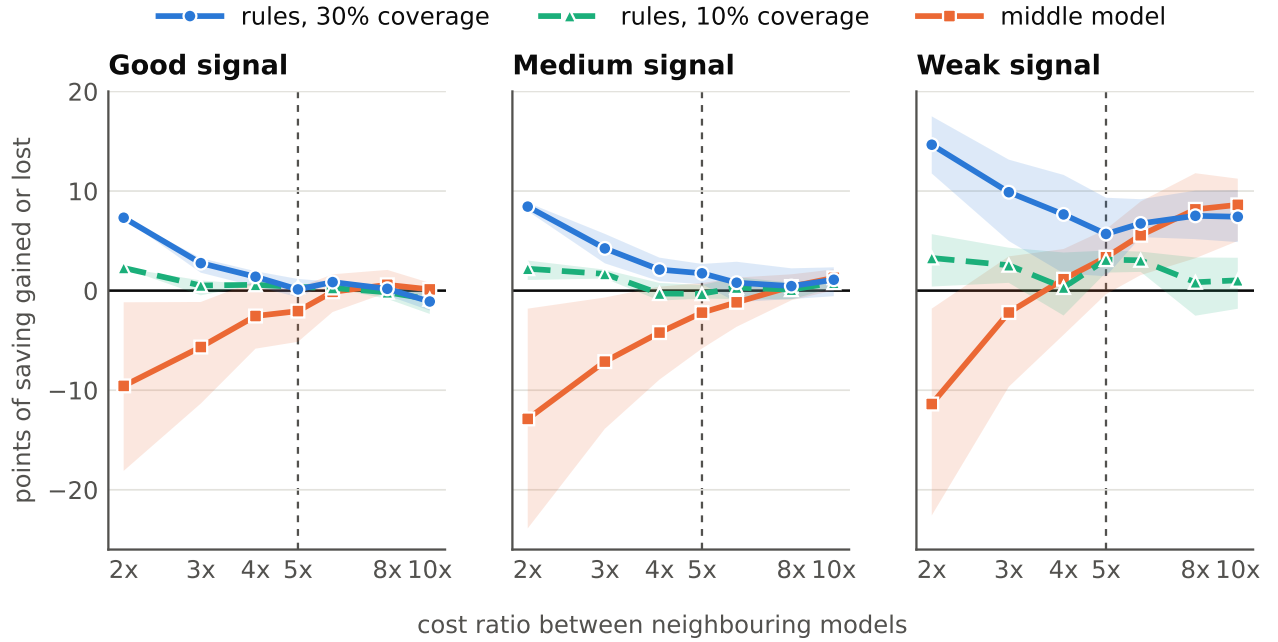


Figure 2: Points of saving gained or lost by a third stage, against a two-stage cascade. Line: mean over three shares of easy queries (50%, 70%, 90%). Band: minimum to maximum.

4.3 Rules as the third stage

Figure 2 puts a rule stage next to the middle model, on the same queries.

Two things stand out. First, the rule stage is safe: over all 126 cells its worst result is a loss of about 3 points, against 24 for the middle model. Second, what it gains depends on how much it covers and on how close the models are. With 30% coverage and models less than 5x apart it gains 0 to 18 points; with 10% coverage, -2 to +6.

The mechanism is simple. The rule stage removes easy queries before the small model sees them. When models are close in cost, the small model’s own compute is a meaningful share of the total, and skipping it matters. When the signal is weak, the rule stage also removes queries that the small model would have escalated by mistake. That is why the largest gains appear with a weak signal: 2 to 18 points at 30% coverage below 5x.

4.4 The 5x rule

Table 2 summarises both comparisons around a single threshold.

Table 2: Range of the gain in points of saving, against a two-stage cascade.

Third stage	Below 5x	5x or more
Rule stage, 30% coverage	0 to +18	-2 to +10
Rule stage, 10% coverage	-2 to +6	-3 to +4
Middle model, weak signal	-23 to +4	0 to +15
Middle model, good or medium signal	-24 to +1	-6 to +4

The rule of thumb that follows:

1. **Under 5x between neighbouring models, the third stage is rules, not a model.** A middle model usually costs more than it saves.
2. **From 5x, with a weak confidence signal, add a middle model.** Rules still help if they cover enough queries.
3. **From 5x, with a good or medium signal, two models are enough.**
4. **With few easy queries, do not build a cascade.**

The edges are soft. At 3x to 4x, with a weak signal and mostly easy queries, a middle model already gains up to 4 points; at 5x with a weak signal and only half the queries easy, it breaks even. The threshold is a round number chosen to be remembered. What the data support is the direction: the closer the models, the less a middle model is worth, and the more a rule stage is.

4.5 Workload profiles

Table 3 applies the planner to five illustrative profiles. They are assumptions about typical workloads, not measurements. The planner recommends the simplest design within 2 points of the best: fewest models first, then no rule stage.

Table 3: Recommended design and its saving for five illustrative workload profiles.

Workload (easy, rule coverage, signal)	Costs 1 : 3 : 9	Costs 1 : 8 : 70
Ticket triage (80%, 30%, good)	rules, small, large: 74%	small, large: 81%
Catalogue matching (70%, 50%, good)	rules, small, large: 67%	small, large: 72%

Workload (easy, rule coverage, signal)	Costs 1 : 3 : 9	Costs 1 : 8 : 70
Assistant turns (60%, 10%, weak)	rules, small, large: 33%	rules, small, medium, large: 48%
Document extraction (50%, 20%, weak)	rules, small, large: 29%	small, medium, large: 39%
Open-ended analysis (10%, 0%, weak)	large model only	small, large: 5%

With close costs the recommendation is the same for every profile that has easy queries: rules in front of two models. With far-apart costs it depends on the signal, as the rule predicts.

5. Keeping a cascade honest

A cascade is only as good as its calibration, and calibration decays as the workload changes. Two chi-square tests make the decay visible.

Calibration. Group answered queries into bins of declared confidence. With p_b the mean declared confidence, o_b the observed frequency of correct outcomes and n_b the number of cases in bin b ,

$$\chi^2 = \sum_b \frac{n_b (o_b - p_b)^2}{p_b (1 - p_b)}$$

is compared with a chi-square distribution with as many degrees of freedom as bins. A large value means the declared confidence no longer matches reality and the cascade must be recalibrated.

Drift. With observed and expected counts per category of query,

$$\chi^2 = \sum_k \frac{(O_k - E_k)^2}{E_k}$$

flags a change in the mix of queries, and each category's share of the statistic shows where the change is. For example, expected counts of 160, 100, 100, 40 and observed counts of 130, 110, 120, 40 give 10.63, above the 5% critical value of 7.81 for three degrees of freedom, with more than half of the statistic coming from the first category.

Both tests need outcomes: operator corrections, confirmed orders, validation rules, sampled review. A workload with no outcome data cannot be calibrated, and should not be routed by confidence.

6. Planning a workload

The rule and the simulation are packaged as a planner. It takes four numbers:

1. **The share of easy queries.** Run the smallest model on a sample with known answers; this is the share it gets right with high confidence.
2. **Rule coverage.** The share of queries a deterministic step can answer outright.
3. **The cost ladder.** Measured compute per query for each available model.
4. **The quality of the confidence signal.** The area under the ROC curve of the small model's confidence against right and wrong answers.

It returns every design with its simulated saving and recommends the simplest one within 2 points of the best. The same planner runs in the browser as a single self-contained page, in English and Italian. The output is a what-if under the simulation's assumptions: it tells a team which designs are worth measuring, not what a real system will save.

7. Related experiments in the repository

The repository applies the same funnel logic to a second problem: choosing the best of many noisy candidates under a fixed evaluation budget. The main results, all simulated:

- Three elimination stages raise the probability of picking the truly best of 1,000 candidates from 21% to 68% on noisy data, at the same budget. More than six stages make things worse.
- Cutting lightly first and harder later is the robust schedule. No named number sequence does better than a generic one growing about 1.4 times per stage.
- A cascade of cheap, medium and precise evaluators wins by a wide margin on a tight budget (69% against 21%), and loses to the precise evaluator alone on a generous one.
- Measurements of different quality should be combined with inverse-covariance weights, never averaged.

These are consistent with the literature on staged elimination [8-12] and are not claimed as new.

8. Limits

- **Synthetic data.** Difficulty, ability and the confidence signal follow simple Gaussian assumptions.
- **The simulated signals are informative.** Even the weak signal has an area under the ROC curve of 0.86 to 0.94 for the small model. Confidence signals of real language models are often weaker than that. The boundaries reported here may move, in either direction, once real signals are used.
- **Independent errors.** Each model’s confidence noise is independent. Models of the same family may fail on the same queries, which would reduce the value of escalation.
- **An idealised rule stage.** It is 99.5% accurate and fires only on easy queries. Real rules misfire, and their value falls as they do.
- **Assumed costs.** Cost ladders are inputs, and regular ones ($1 : r : r^2$). Real ladders are irregular.
- **A coarse threshold grid.** The threshold is chosen from ten values. Some cell-to-cell variation of a point or two comes from the grid, not from the designs.
- **Memory is not counted.** A cascade keeps every model loaded. On a single GPU a smaller model takes memory from the larger one.
- **Illustrative profiles.** The workloads in table 3 are assumptions, not measured systems.

9. Testing the rule on real models

The protocol, in the repository, is designed so that every design can be evaluated offline from a single pass:

1. Take three open-weight models of clearly different size, served locally, and a public benchmark with known answers, split once into a calibration half and a test half.
2. Run all three models on every item once. Store the answer, whether it is correct, the confidence signal and the measured cost.
3. Fit calibration and choose the threshold on the calibration half.
4. Evaluate every design on the test half, with bootstrap confidence intervals.

The comparisons are the large model alone, two stages against three, a rule stage where the task allows one, and model sets with close and with far-apart costs.

What would count against the rule:

- a third stage, middle model or rules, never beating two stages at equal accuracy;
- the boundary not appearing where the cost ratio and the signal quality say it should;
- confidence signals too weak for any threshold to hold accuracy;
- savings that vanish once the measured cost of running the smaller models first is included.

Negative results will be published as they are.

10. Conclusion

Two-tier cascades with calibrated confidence are established. This report addresses the step after: whether to add a third stage, and which one. In simulation the answer depends mostly on one number, the cost ratio between neighbouring models. Under 5x a middle model usually loses and a rule stage gains in proportion to what it covers; from 5x a middle model pays only when the confidence signal is weak. The best third stage is often not a model.

This is a hypothesis with code attached. The most useful next contributions are a run of the protocol on real models and a measured use case: the four numbers of a real workload, and what the cascade actually saved.

Reproducibility

Code, results and this report, in English and Italian, are at <https://github.com/gionnysimonetti-dev/calibrated-funnel> under the MIT License. `python -m experiments.run_all` regenerates every number in about five minutes; seeds are fixed per experimental cell. Tables 1 to 3 and figures 1 and 2 correspond to experiments 1, 6, 7 and 8.

Use of AI

The simulation code and part of the statistical analysis were developed with an AI assistant (Claude, Anthropic), which also assisted in drafting this report. The author reviewed the design and takes responsibility for the content.

References

1. L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.
2. V. Kotte. UCCI: Calibrated Uncertainty for Cost-Optimal LLM Cascade Routing. arXiv:2605.18796, 2026.
3. Y. Dou, S. Lian, S. Li. Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference. arXiv:2607.25018, 2026.
4. A. Barrak, Y. Fourati, M. Olchawa, E. Ksontini, K. Zoghalmi. CARGO: A Framework for Confidence-Aware Routing of Large Language Models. arXiv:2509.14899, 2025.
5. Y.-N. Chuang, P. K. Sarma, P. Gopalan, J. Boccio, S. Bolouki, X. Hu, H. Zhou. Learning to Route LLMs with Confidence Tokens. arXiv:2410.13284, 2024.
6. C. Hao, W. Lu, Y. Ishiwaka, Z. Li, W. Wan, Y. Chen. When Models Know When They Do Not Know: Calibration, Cascading, and Cleaning. arXiv:2601.07965, 2026.
7. P. Aggarwal, A. Madaan, et al. AutoMix: Automatically Mixing Language Models. arXiv:2310.12963, 2023.
8. Z. Karnin, T. Koren, O. Somekh. Almost Optimal Exploration in Multi-Armed Bandits. ICML, 2013.
9. K. Jamieson, A. Talwalkar. Non-stochastic Best Arm Identification and Hyperparameter Optimization. AISTATS, 2016.
10. S. Shahrampour, M. Noshad, V. Tarokh. On Sequential Elimination Algorithms for Best-Arm Identification in Multi-Armed Bandits. arXiv:1609.02606, 2016.
11. L. Li et al. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. JMLR, 2018.
12. W. G. Cochran. Improvement by means of selection. Proceedings of the Second Berkeley Symposium, 1951.
13. C. Guo et al. On Calibration of Modern Neural Networks. ICML, 2017.

14. D. W. Hosmer, S. Lemeshow. Goodness-of-fit tests for the multiple logistic regression model. *Communications in Statistics*, 1980.